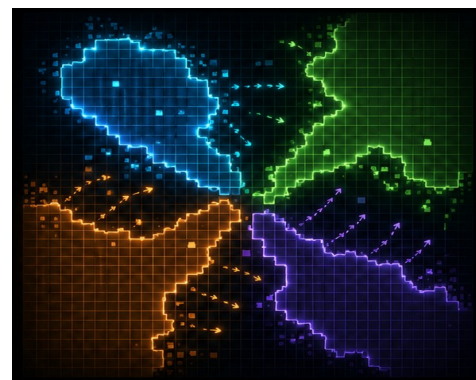




v1.0 P.G.L. Cellules 2D

FILIERE ING1-GI • 2025-2026

AUTEURS E.ANSERMIN - R.GRIGNON

E-MAILS eva.ansermin@cyu.fr - romuald.grignon@cyu.fr

DESCRIPTION

- Dans ce projet vous devez créer une application graphique qui permette de simuler le comportement de cellules dans un plan 2D. Ces cellules doivent pouvoir évoluer à la fois en fonction du temps (naissance, mort, déplacement, taille, ...), en fonction de l'environnement (cellules aux alentours, interactions, ...) et enfin en fonction de l'utilisateur lui même qui peut modifier les caractéristiques individuelles des cellules (renforcement, destruction, ...).
- Le but est de pouvoir interagir visuellement sur cette carte 2D et d'observer les modifications en direct.
- Le périmètre technique est détaillé dans ce document. Le service rendu d'un point de vue utilisateur doit, quant à lui, être en adéquation avec le périmètre Ethique/Design sur lequel vous avez travaillé pendant les semaines précédentes. Ce périmètre devra être fourni à votre tuteur de projet (livrables Ethique/Design) pour qu'il puisse vérifier la cohérence.

INFORMATIONS GENERALES

- **Taille de l'équipe**
Ce projet est un travail d'équipe. Il est préférable de se réunir en groupe de 5 personnes. Si le nombre d'étudiants n'est pas un multiple de 5 et/ou si des étudiants n'arrivent pas à constituer des groupes, c'est aux chargés de projet de statuer sur la formation des groupes. Pensez donc à anticiper la constitution de vos groupes pour éviter des décisions malheureuses.
- **Démarrage du projet et jalons**
Vous obtiendrez de plus amples informations quant aux dates précises de rendu, de soutenance, les critères d'évaluation, le contenu du livrable, ..., quand le projet démarrera officiellement. Quel que soit le planning initial du projet, vous veillerez à **planifier** plusieurs **rendez-vous** d'avancement avec votre chargé(e) de projet. C'est à votre groupe de

prendre cette **initiative**. L'idéal est de faire un point 1 ou 2 fois par semaine mais cette fréquence est laissée libre en fonction des besoins identifiés avec le tuteur de projet.

➤ **Versions de l'application**

Pour éviter d'avoir un projet non fonctionnel à la fin, il vous est demandé d'avoir une version en ligne de commande fonctionnelle. Ceci vous permettra de tester votre modèle de données indépendamment de l'interface graphique. C'est la version avec l'interface graphique **JavaFX** qui sera bien entendu **évaluée** mais dans le cas où certaines fonctionnalités ne seraient pas visibles avec cette interface, vous devez pouvoir présenter toutes les fonctionnalités de votre code Java en ligne de commande.

➤ **Dépôt de code**

Vous devrez déposer la **totalité** des fichiers de votre projet sur un dépôt central **Git**. Il en existe plusieurs disponibles gratuitement sur des sites web comme github.com ou gitlab.com. La fréquence des **commits** sur ce dépôt doit être au minimum de **1 commit/jour**.

➤ **Rapport du projet**

Un rapport écrit est requis, contenant une brève description de l'équipe et du sujet. Il décrira les différents problèmes rencontrés, les solutions apportées et les résultats. L'idée principale est de montrer comment l'équipe s'est organisée, et quel était le flux de travail appliqué pour atteindre les objectifs du cahier des charges. Le rapport du projet peut être rédigé en français ou en anglais. Ce rapport contiendra en plus les éléments techniques suivants : un document de conception UML (diagramme de classe) de votre application, et un document montrant les cas d'utilisations.

➤ **Démonstration**

Le jour de la présentation de votre projet, votre code sera exécuté sur vos machines. Prévoyez d'amener **2 machines** fonctionnelles. Il sera possible que le jury vous demande d'effectuer des modifications en direct de votre code, pendant que vous présentez le fonctionnel de votre application d'où la nécessité d'avoir ces 2 machines.

Si le jury vous demande de faire des modifications, il sera surement nécessaire de le faire sans l'utilisation de l'IA, il est donc important que vous maîtrisiez votre code ce jour là.

La version utilisée sera la **dernière** fournie sur le **dépôt** Git avant la date de rendu. Même si vous avez une nouvelle version qui corrige des erreurs ou implémente de nouvelles fonctionnalités le jour de la démonstration, c'est bien la version du rendu qui sera utilisée.

Si vous avez une version **livrée en retard** et que vous souhaitez qu'elle soit utilisée lors de la présentation, il faudra le préciser **par écrit à l'avance**, afin que nous puissions en tenir compte de ce retard dans la notation.

Le jour de la présentation, vous devrez, en **ligne de commande**, récupérer votre dépôt Git, compiler votre projet, et le lancer. Il sera **interdit** de faire le test de votre application via un environnement de développement.

➤ **Organisation de l'équipe**

Votre projet sera stocké sur un dépôt git (ou un outil similaire) tout au long du projet pour au moins trois raisons :

- éviter de perdre du travail pendant le développement
- être capable de travailler en équipe efficacement
- partager vos progrès de développement facilement avec votre chargé(e) de projet.

De plus il est **recommandé** de mettre en place un environnement de travail en équipe (Slack, Trello, Discord, ...).

CRITERES DE NOTATION

-
- Le but principal du projet est de fournir une **application fonctionnelle** pour l'utilisateur. Le programme doit correspondre à la description en début de document et implémenter toutes les fonctionnalités listées.
 - Votre application devra reprendre autant que possible le **périmètre Ethique et Design** sur lesquels vous avez travaillés au préalable.
 - Votre projet doit être utilisable au clavier ou à la souris en fonction des fonctionnalités nécessaires.
 - Tous les éléments de votre code (variables, classes, noms de fichiers ou dossiers, fonctions, commentaires) seront écrits **en langue anglaise** obligatoirement.
 - Votre code devra être **commenté** de telle manière que l'on puisse utiliser un outil de génération de documentation automatique de type **JavaDoc**. Un dossier contenant la documentation générée par vos soins sera présent dans votre dépôt de code lors de la livraison.
 - Votre application ne doit jamais s'interrompre de manière intempestive (crash), ou tourner en boucle indéfiniment, quelle que soit la raison. Les exceptions qui s'affichent sur la console sont des Exceptions qui n'ont pas été gérées par votre code et sont bien considérées comme des crash (même si dans le cas d'une Application JavaFX, le processus d'affichage graphique est séparé et donc que la fenêtre reste ouverte). Une Exception non gérée sera très pénalisant sur la notation.
 - Toutes les erreurs doivent être gérées correctement. Il est préférable d'avoir une application stable avec moins de fonctionnalités plutôt qu'une application complète mais qui plante trop souvent.
 - Votre application devra être organisée en modules afin de ne pas avoir l'ensemble du code dans un seul et même fichier par exemple. Apportez du soin à la conception de votre projet avant de vous lancer dans le code.
 - Le livrable fourni à votre tuteur de projet sera simplement l'URL de votre dépôt Git accessible publiquement à la date du rendu. **Inutile** d'envoyer des **fichiers** par **email** ou **messagerie** instantanée : seuls les fichiers

présents dans le dépôt à la date du dernier commit antérieur à la date limite seront pris en compte. Si vous décidez de livrer des fichiers oubliés ou modifiés après la date limite, et que vous insistez pour que cette version soit prise en compte, alors vous aurez des **pénalités de retard** qui s'appliqueront et elles seront très punitives sur la note finale. Prévoyez vos tests et votre livraison à l'avance.

- Toutes les fonctionnalités décrites dans la suite du document sont purement techniques, mais le but principal d'une application est de répondre à une problématique d'un utilisateur : il faut donc adapter ce cahier des charges pour répondre à votre périmètre Ethique/Design. De fait, les statistiques, les actions possibles, les affichages, ..., doivent être adaptées le plus possible à votre projet personnel.

DETAILS DU PROJET

➤ Format de la grille

La grille de simulation doit être configurable en dimensions. Sa topologie peut être bornée, mais aussi torique (les bords extérieurs se rejoignent).

➤ Cellules

Chaque cellule dans la grille doit posséder des caractéristiques individuelles comme par exemple :

- mode/vitesse de déplacement
- énergie/vie/âge et conditions de perte/gain
- état (actif, dormant, ...)
- résistances/faiblesses
- rayon d'influence/de perception
- modes d'interactions avec l'environnement et conséquences

➤ Interaction avec la grille

L'application doit permettre de créer/modifier/supprimer des cellules individuellement. Il existe plusieurs manières de le faire par exemple :

- mode « pinceau » : l'utilisateur peut remplir la grille avec un archétype de cellule pré-défini en utilisant la souris
- mode « zone » : l'utilisateur sélectionne une zone rectangulaire pour remplir les cases de la grille
- mode individuel : l'utilisateur clique sur chaque case qu'il souhaite modifier

Il est possible de faire des ajouts de masse avec des positions et des propriétés aléatoires : les plages de valeurs doivent être réglables sur l'application.

Il est possible de combiner les 2 fonctionnalités précédentes.

➤ Simulation

La simulation doit posséder un réglage de vitesse (le pas de simulation). Ainsi l'utilisateur peut faire évoluer la grille à son rythme. Dans ce réglage, un mode « step-by-step » doit être possible, ainsi que des boutons pour mettre en pause, et reprendre la simulation.

A chaque étape de la simulation, chaque case va évoluer en fonction des règles que vous aurez établies, basées sur les propriétés des cellules, le temps, et sur leur environnement. Il faut nécessairement les fonctionnalités suivantes (pas forcément pour toutes les cellules) :

- déplacement
- division/fusion/propagation cellulaire
- interaction/combat/renforcement/affaiblissement par contact ou proximité avec d'autres cellules
- mort en fonction du temps

Les modes d'interaction *peuvent* être multiples (4 voisins orthogonaux, 8 voisins directs, distance euclidienne/manhattan, ...) pour montrer différent comportements.

Pour tous les modes d'interaction, ainsi que toutes les propriétés qui dépendent du temps, des notions de probabilités doivent être incluses : ces probabilités peuvent être globale au système, ou propres à chaque cellule.

Des statistiques sur la grille totale doivent être affichées en direct comme par exemple :

- nombre / % de cellules de chaque type
- tendance de progression à la hausse ou baisse de chaque type
- statistiques sur les propriétés (age moyen, énergie min/max, ...) via des graphiques (histogrammes, courbes, ...)

➤ Import/Export

A n'importe quel moment de la simulation, l'utilisateur peut choisir de la sauvegarder au format binaire sur le disque dur.

Il faut donc proposer un moyen de restaurer un fichier de simulation pour reprendre son execution là où elle s'était arrêtée.

RESSOURCES UTILES

GitHub

- <https://www.github.com>
- <https://docs.github.com/en/get-started/quickstart/hello-world>

Patrons de conception

- https://fr.wikipedia.org/wiki/Patron_de_conception

Sérialisation

- <https://fr.wikipedia.org/wiki/Sérialisation>

Automates cellulaires

- https://fr.wikipedia.org/wiki/Automate_cellulaire